

Inteligência Artificial - Trabalho Prático

Cátia Eira (a107382), Filipa Cosquete (a106837), Marco Ferreira (a106857)

Grupo 34

2025 / 2026



Avaliação dos pares

a107382 Cátia Eira DELTA = 0

a106837 Filipa Cosquete DELTA = 0

a107857 Marco Ferreira DELTA = 0

Índice

1. Descrição do problema.....	4
2. Formulação.....	5
3. Trabalho Realizado.....	6
3.1. Representação sob a forma de grafo.....	6
3.2. Estratégias de Procura.....	7
3.3. Heurísticas.....	7
3.4. Comparação de algoritmos ponto-a-ponto.....	8
3.5. Heurística Alternativa.....	8
3.6. Procuras multi-destino.....	9
4. Simulação.....	11
4.1. Carro.....	11
4.2. Controlador do carro.....	11
4.3. Tarefas.....	11
4.4. Fazer viagem do cliente.....	11
4.5. Fatores dinâmicos.....	12
5. Análise de resultados.....	13
6. Conclusão.....	14

1. Descrição do problema

O objetivo deste projeto é otimizar o custo de operação de um sistema de gestão de táxis, assegurando simultaneamente a satisfação das necessidades dos clientes e a prestação do melhor serviço possível. Para tal, pretende-se garantir que cada cliente seja atendido no menor tempo de espera viável, de acordo com as suas preferências ambientais e se é premium. O sistema deve ser capaz de operar de forma rápida e eficiente, mantendo um desempenho ótimo mesmo em cidades de grande dimensão.

Devem ser consideradas as diferenças entre veículos a combustão e veículos elétricos, nomeadamente no que diz respeito à capacidade, autonomia e custos operacionais, de modo a selecionar o veículo mais adequado para cada pedido de cliente. Adicionalmente, o sistema deve determinar de forma autónoma quando um veículo necessita de abastecimento ou carregamento, assegurando que este nunca fica imobilizado por falta de energia, tendo em conta apenas estações compatíveis com o respetivo tipo de veículo.

O sistema deverá ainda reagir dinamicamente às condições de trânsito, recalculando rotas sempre que necessário para garantir que o percurso em curso continua a ser o mais eficiente até ao destino. Por fim, devem estar disponíveis informações globais sobre o funcionamento do sistema, tais como o tempo médio de espera dos clientes, o custo operacional acumulado e outros indicadores relevantes para a avaliação do desempenho.

2. Formulação

Na ausência de estados dinâmicos, nomeadamente a criação e destruição aleatória de veículos e clientes, bem como a variação das condições de trânsito, o problema pode ser classificado como um problema de múltiplos estados. Neste cenário, o ambiente é totalmente observável, uma vez que o sistema tem acesso à localização de todos os veículos, clientes e à totalidade do mapa, e o seu comportamento é determinístico.

Quando os estados dinâmicos estão ativados, configuração que mais se aproxima da realidade e que é assumida como a opção padrão neste trabalho, o problema passa a enquadrar-se na categoria de problemas de contingência, dado que o estado do sistema pode variar de forma imprevisível ao longo do tempo.

Um estado do sistema é definido pelo tempo atual, pelas condições de trânsito em vigor, pelas posições e características operacionais dos veículos, bem como pelo conjunto de clientes e respetivos pedidos. O estado inicial caracteriza-se pela ausência de clientes e veículos, assumindo-se, para todas as arestas do mapa, velocidades correspondentes ao limite máximo permitido.

As principais transições de estado resultam da geração de pedidos por parte dos clientes e da execução das viagens pelos veículos. Quando um pedido é efetuado, um veículo elegível inicia o deslocamento até ao cliente e, posteriormente, até ao destino final. Para que esta transição ocorra, é necessário que exista pelo menos um cliente e um veículo com capacidade suficiente, um caminho viável entre o veículo, o cliente e o destino, bem como autonomia energética para completar toda a viagem. O efeito desta ação traduz-se na satisfação do cliente e na atualização do estado do veículo, incluindo a redução do nível de combustível ou carga elétrica, o aumento da quilometragem total, entre outros parâmetros relevantes.

O custo associado ao problema pode ser definido com base no tempo despendido em cada viagem ou no custo operacional, calculado a partir da distância percorrida e dos custos específicos de cada veículo. O sistema permite a definição de diferentes preferências de otimização; contudo, por defeito, privilegia-se a minimização do tempo de viagem para o cliente e a minimização da distância nas restantes deslocações operacionais.

3. Trabalho Realizado

3.1. Representação sob a forma de grafo

De forma a obter um grafo o mais fiel à realidade possível, foi usada a biblioteca OSMnx, que permite obter grafos NetworkX (que representam mapas do Open Street Map) relativos a qualquer localidade ou cidade. Obtido o grafo, é feita uma conversão para o nosso tipo de dados para representar grafos, que é constituído por:

- um dicionário de nós (a chave é o id do nó) - dict[str, Node];
- um dicionário de listas de adjacência (a chave também é o id do nó), que guarda para cada vizinho o id do nó e um objeto EdgeData que contém os restantes dados necessários - dict[str, list[tuple[str, EdgeData]]];
- um dicionário de listas de adjacência invertidas do mesmo formato do anterior (usado para otimizar as procuras, como será explicado na secção relativa aos algoritmos utilizados);
- a velocidade máxima que ocorre no grafo (utilizada para o cálculo da heurística);
- dois dicionários para fazer caching dos valores de heurísticas já calculadas (distâncias e tempos), evitando assim recalcular desnecessariamente - dict[tuple[str,str], float]

Já as informações relativas aos nós são guardadas em objetos da classe Node, sendo elas:

- id do nó;
- tipo do nó, i.e., se o nó é um posto de abastecimento, uma estação de recarga, ambos ou nenhum;
- coordenadas geográficas (usadas para calcular as heurísticas, como se descreve mais adiante).

Por fim, as informações relativas às arestas são guardadas em objetos EdgeData, permitindo assim deduplicar os dados das arestas nos dois dicionários de listas de adjacências. Esses dados são:

- distância da porção de via representada pela aresta;
- limite de velocidade a que se pode circular na via em questão;
- velocidade atual a que se pode circular (menor do que a máxima, representando o trânsito);
- multiplicador (valor menor ou igual a 1 usado para calcular a velocidade atual a partir da máxima, simulando trânsito).

Mais detalhes sobre a introdução de trânsito são dados nas secções mais à frente.

3.2. Estratégias de Procura

Com o objetivo de realizar procuras no mapa, foram desenvolvidas diversas estratégias com base em algoritmos de procura em grafos, desde procuras ponto-a-ponto até procuras multi-destino com restrições, considerando tanto distâncias como tempos de travessia.

Portanto, começa-se por apresentar uma comparação entre os algoritmos de Dijkstra, Greedy e A*, de maneira a perceber qual a melhor opção para as nossas necessidades. No entanto, é relevante descrever primeiro as heurísticas utilizadas.

Nota: de modo a obter resultados mais fiáveis e numa escala aceitável, foi utilizado o grafo da cidade do Porto (175083 nós e 397036 arestas) para executar todos os testes apresentados nesta secção, sendo que o resultado de todas as procuras é um caminho com 300 a 400 nós. Todos os tempos apresentados são resultado de uma média aritmética de 5 execuções. Para os algoritmos que envolvem heurísticas, não foi considerada a primeira execução, pois os valores das heurísticas são todos calculados do zero; da segunda em diante, estes passam a estar em cache.

3.3. Heurísticas

Para as procuras por distância, a heurística utilizada é a distância em linha reta entre dois nós, calculada através da fórmula de haversine (usada para calcular a distância entre dois pontos de uma esfera a partir das suas latitudes e longitudes; assim, é possível considerar a curvatura da Terra). Já para as procuras por tempo, a heurística é praticamente a mesma, sendo apenas considerada a velocidade máxima que ocorre no grafo para calcular o tempo que demora percorrer a distância em linha reta. Desta forma, tem-se uma heurística admissível e consistente, garantindo a otimalidade das procuras A*. Para que uma heurística seja admissível, esta nunca deve superestimar o verdadeiro custo; para que seja consistente, quando há uma deslocação para um nó vizinho, o valor da heurística só pode diminuir, no máximo, o custo verdadeiro dessa deslocação.

- **Admissibilidade** - A distância em linha reta é sempre menor ou igual à verdadeira distância da deslocação. Isto porque as estradas ou são mesmo completamente retas (situação em que a propriedade ainda se verifica) ou têm curvas, o que faz aumentar a distância real. No caso das procuras por tempo, ao considerar a velocidade máxima que ocorre em todo o grafo, também se garante que o valor da heurística nunca ultrapassa o tempo real, embora a estimativa possa ser demasiado forte (por exemplo, se a velocidade máxima no grafo for 120 km/h e estiver a ser feita uma procura que só envolve arestas de 50 km/h, o valor da heurística será bastante baixo). No entanto, nunca se viola a propriedade.
- **Consistência** - Para as distâncias, sempre que se faz uma deslocação em direção ao destino, o valor mínimo que a distância real pode diminuir é o mesmo que a distância em linha reta, pelo que a diminuição da heurística nunca será maior do que a diminuição do custo verdadeiro da deslocação. Quanto às procuras por tempo, fazendo os cálculos considerando a velocidade máxima que ocorre no grafo, também se garante esta propriedade.

3.4. Comparação de algoritmos ponto-a-ponto

As seguintes comparações foram feitas com a procura por tempo, onde a heurística é o tempo que demora a percorrer a distância em linha reta à velocidade máxima possível no grafo, conforme descrito na secção anterior. Os resultados estão no formato <tempo de execução (milissegundos)> | <solução da procura (segundos)>. A última linha da tabela é discutida na secção Heurística Alternativa.

	238143566 -> 239094017	5124119101 -> 239253414	238143566 -> 8455633803
Dijkstra	910 5318	865 5458	710 3501
Greedy	6 7491	6 8592	6 5878
A*	914 5318	688 5458	522 3501
A* alt	237 5346	108 5458	71 3529

Tabela 1 - Resultados das procuras ponto-a-ponto

Como era esperado, o algoritmo de Dijkstra obtém sempre a melhor solução, mas demora mais do que os restantes (exceto no primeiro teste, sendo o porquê discutido também na secção Heurística Alternativa), devido à exploração exaustiva de nós. Por oposição, a procura Greedy executa sempre muito rapidamente, pois, guiada pela heurística, explora um único caminho até à solução. No entanto, o resultado da procura não é ótimo, pelo que não se adequa às nossas necessidades. Por fim, o algoritmo A* apresenta um equilíbrio entre os dois anteriores, melhorando o tempo de execução mas sem sacrificar a otimalidade. Esta última característica deve-se ao facto de a heurística utilizada ser admissível e consistente.

Portanto, considerando os resultados obtidos, conclui-se que, dadas as circunstâncias do problema, o melhor algoritmo para procuras ponto-a-ponto é o A*.

3.5. Heurística Alternativa

Conforme evidenciado anteriormente, na primeira procura o algoritmo de Dijkstra apresenta um tempo de execução muito semelhante ao A*. Isto acontece porque, ao longo da procura, o valor do custo acumulado cresce e o valor da heurística diminui, e numa procura longa como esta, isto torna-se muito mais aparente. Portanto, a partir de certo ponto, a heurística deixa de ser significativa na comparação de caminhos, fazendo com que o A* degenera para um Dijkstra, tendo ainda o ligeiro custo computacional associado às heurísticas.

Considerando esta limitação, foram feitas experiências com uma nova heurística para as procuras por tempo: em vez de considerar a velocidade máxima que ocorre no grafo, considerar a velocidade máxima dos vizinhos do nó de origem. Por exemplo, considere-se um grafo com velocidade máxima 120 km/h e dois quaisquer nós A e B, sendo que A tem 3 vizinhos. As arestas que ligam A a dois deles têm velocidade 50 km/h, mas a terceira tem 90 km/h. Desta forma, ao calcular a heurística entre A e B, usa-se a velocidade 90 km/h para calcular o tempo que demora percorrer a distância em linha reta. Com esta

alteração, é possível tornar a heurística mais significativa no final das procuras, uma vez que considerar sempre a velocidade máxima de todo o grafo é, em alguns casos, demasiado otimista.

Como é possível verificar pelos resultados da Tabela 1, o tempo de execução das procuras é, em média, 6 vezes menor do que o A* com a heurística original. No entanto, embora a segunda procura tenha obtido o resultado ótimo, o mesmo não se verificou na primeira e na terceira. Esta ligeira diferença deve-se ao facto de a nova heurística não satisfazer a propriedade da consistência. Considerando novamente o exemplo anterior, e considerando C como o nó vizinho de A ligado pela aresta com velocidade 90 km/h, é possível que todos os vizinhos de C estejam conectados por arestas de velocidade reduzida, como 30 km/h. Considerem-se também as seguintes distâncias:

- Distância em linha reta A–B: 9 km
- Distância em linha reta C–B: 8 km
- Comprimento da aresta A–C: 1 km

Então, $h(A) = (9 / 90) * 3600 = 360$ segundos e $h(B) = (8 / 30) * 3600 = 960$ segundos. Verifica-se assim que, com esta heurística, é possível que o valor de $h(n)$ para dois nós A e C seja maior do que para A e B, com C mais próximo de B do que A, quebrando assim a consistência.

Portanto, optou-se por não utilizar esta nova heurística, uma vez que para o presente problema não se justifica, do nosso ponto de vista, sacrificar a otimalidade em benefício do tempo de execução.

3.6. Procuras multi-destino

A certo ponto do desenvolvimento da simulação, surgiu a necessidade de ter um algoritmo para encontrar a estação (abastecimento/carregamento) mais próxima a partir de um determinado nó. Utilizando apenas procuras ponto-a-ponto, seria necessário executar uma procura para cada par (origem, estação), o que seria muito demorado, considerando que o número de estações não é desprezável. Portanto, surgiu a ideia de executar apenas uma procura a partir da origem, parando-a quando se encontra uma estação do tipo pretendido, em vez de parar ao encontrar o destino em concreto como numa procura ponto-a-ponto. Para este caso, optamos por utilizar o algoritmo de Dijkstra, pois as estações são distribuídas pelo grafo de maneira uniforme e minimamente abundante, tornando a expansão “circular” mais adequada para encontrar a estação mais próxima. Por outras palavras, usar A* para ter direção na procura não é impactante devido à distribuição das estações.

Outra necessidade que surgiu foi a de encontrar o carro mais adequado (a nível de tempo ou custo operacional) para atender um cliente. Começou-se por fazer uma procura A* para cada par (carro, cliente), comparando os N resultados. Posteriormente, surgiu a ideia de fazer algo semelhante à procura de estações: inverter o sentido da procura, isto é, em vez de procurar N vezes no sentido carros -> cliente, procurar apenas uma vez no sentido cliente -> carros, tendo-se começado por implementar este algoritmo com Dijkstra. Para isso, houve a necessidade de armazenar as arestas invertidas num segundo dicionário de listas de adjacência. Assim, os restantes algoritmos usam as listas de adjacência normais enquanto que as procuras por carros usam as adjacências invertidas.

Porém, acontece que o resultado obtido foi inferior às N procura A*. Isto porque, ao contrário do caso das estações, a distribuição dos carros pelo mapa é não uniforme e em menor quantidade, pelo que se torna interessante usar as heurísticas para guiar a procura. Sendo assim, trocou-se o Dijkstra pelo A*, utilizando para as heurísticas o mínimo entre os carros, ou seja, ao avaliar um nó, calcula-se o mínimo do valor de $h(n)$ para todos os pares (posição de carro, cliente). Desta forma, é possível garantir a admissibilidade e a consistência da heurística, mantendo assim as garantias de otimalidade do A*. O pior caso para este algoritmo é quando não existe um caminho do cliente para um dos carros, situação em que tem um comportamento igual ao Dijkstra. Concluindo, fazer uma só procura A* no sentido invertido é, teoricamente, mais vantajoso.

Passando à prática, os resultados a seguir apresentados foram obtidos com $N = 4$ (ou seja, para as linhas 2 e 3, 4 destinos).

	Procura de carros por custo operacional
N procura A*	501 ms
Dijkstra multi-destino	677 ms
A* multi-destino	252 ms

Tabela 2 - Tempos de execução de uma procura multi-destino

Em suma, fazer procura multi-destino em vez de N ponto-a-ponto é uma forma de otimizar as travessias nos grafos, pois, devido ao facto de a origem ser a mesma, é possível reutilizar expansões de nós.

4. Simulação

A simulação baseia-se numa iteração constante, onde um incremento simboliza um minuto passado. A cada instante, os controladores dos clientes e carros atualizam todos os seus objetos, baseados nesse “tempo atual”. Cada carro terá uma lista de ‘tarefas’, ordenadas pela sua prioridade, que indicam ao carro o que devem fazer. É possível ativar elementos dinâmicos, de modo a que carros ‘apareçam’ e ‘desapareçam’ se estiverem livres, permitir a clientes aparecerem em nós aleatórios do mapa, e alterar o nível de trânsito, o que afeta as velocidades nas arestas entre os nós.

A decisão de implementar um sistema baseado em *ticks*, ao invés de tempo real, foi feita por questões de simplicidade de implementação e por maior facilidade na demonstração dos algoritmos.

4.1. Carro

Existem dois tipos de carros, carros a combustível e carros elétricos. Estas categorias têm várias diferenças entre si, nomeadamente o tipo de estação que podem reabastecer, o custo de operação e a autonomia. Clientes podem preferir usar apenas carros elétricos.

4.2. Controlador do carro

O controlador dos carros orquestra todos os carros ativos, desde a atribuição dos clientes aos carros certos, à escolha do melhor caminho com base no tempo ou custo operacional. O melhor caminho é escolhido através do algoritmo fornecido, o qual é possível alterar a qualquer momento, a partir do menu. Para fazer a viagem, o carro precisa primeiro de se deslocar até à localização do cliente, e daí até ao destino. Para determinar se um carro é adequado para fazer uma viagem são considerados vários critérios, como a quantidade de combustível ou lugares livres. Clientes premium têm prioridade, e são escolhidos primeiro que os outros clientes. É também possível alterar a prioridade na escolha: menor tempo para fazer a entrega, ou menor distância (menor custo). Quando o controlador termina de calcular o caminho, este irá encarregar o carro de fazer a viagem.

4.3. Tarefas

Tarefas são o que um carro tem que fazer. Todas as tarefas guardam o instante de tempo no qual foram começadas, o tempo estimado que falta (tempo este baseado no contador) e uma *flag* a indicar se estão completas. Se sim, são removidas da lista do carro. A cada atualização, o seu estado será verificado, o que, dependendo do tipo de tarefa, poderá implicar alterações nos controladores, como a remoção de clientes (uma vez dados como satisfeitos). As tarefas estão ordenadas por prioridade, sendo possível interromper as de baixa prioridade (reabastecer quando a autonomia ainda está bastante acima da média de KM percorridos nas viagens, por exemplo), para preferir entrega de clientes.

4.4. Fazer viagem do cliente

Ao fazer a viagem do cliente, o carro terá de ir de nó em nó, movimentos estes que irão variar de acordo com o tempo atual, a distância e a velocidade da estrada (que pode

ser afetada pelo trânsito). Quando acaba a viagem, o carro verifica o seu nível de energia. Se não houver mais viagens para fazer, e tiver pouca energia, ele irá procurar a estação mais próxima para recarregar. Caso contrário, dirige-se à zona mais comum entre os clientes que começaram as viagens.

4.5. Fatores dinâmicos

Existem quatro fatores dinâmicos considerados na simulação, que representam a possibilidade de aparecerem clientes de forma aleatória, aparecerem e desaparecerem carros, de forma a simular um sistema de uber, alterar aleatoriamente os níveis de trânsito, e fazer com que os carros que não estão a efetuar uma entrega de clientes se dirijam a zonas mais centrais do mapa.

Assim, a cada iteração, a probabilidade de aparecer um novo cliente é de 20%. Para os carros, é escolhida uma ação, criar ou eliminar, com 50% de probabilidade cada, e depois, caso seja escolhido criar, existe uma chance de 40% de efetivamente aparecer um carro novo, e, caso contrário, 10% de chance de efetivamente eliminar um carro que não esteja em viagem.

O trânsito tem 20% de probabilidade de ser alterado dinamicamente, atualizando a velocidade de cada aresta para um valor entre 10% e 100% da velocidade máxima permitida em cada uma. Quando o trânsito é alterado, todos os caminhos ativos são recalculados, para procurar um caminho melhor ou evitar continuar no mesmo, caso este tenha piorado.

Quando os carros estão livres e têm combustível suficiente, estes começam a dirigir-se a um determinado ponto mais central entre os clientes. Este ponto é calculado através das coordenadas médias dos pontos de origem dos clientes e associada ao nó mais próximo desta. Esta é uma tarefa de baixa prioridade, pelo que pode ser interrompida por outras.

5. Análise de resultados

Desenvolvida a simulação, foram feitos testes para perceber as diferenças entre fazer as procuras para minimizar o tempo de viagem ou para minimizar o custo operacional. Apresentam-se de seguida os resultados de executar a simulação com fatores dinâmicos e para 1000 iterações, ou seja, 1000 minutos (~17 horas) de tempo real. Foi utilizado o grafo de Braga, que tem 8621 nós e 19741 arestas.

	Minimizando tempo de viagem	Minimizando custo operacional
Total de viagens realizadas	190	199
Total de kms viajados	6313.4	4831.7
Total de kms viajados com clientes	1674.5	1398.5
Custos operacionais (€)	914.2	513.3
Tempo médio por viagem (mins)	25.1	35.2
Tempo médio de espera dos clientes (mins)	7.3	10.5

Tabela 3 - Resultados da simulação

É possível concluir que, minimizando o tempo, o total de quilómetros percorridos é maior, pois minimizar o tempo envolve frequentemente viajar por autoestradas/vias-rápidas. Neste sentido, os custos operacionais são também superiores, uma vez que minimizar os custos operacionais envolve também minimizar os quilómetros percorridos. Por fim, e conforme esperado, os tempos médios de viagem e de espera são inferiores quando se minimiza o tempo.

6. Conclusão

Neste projeto desenvolvemos um sistema de gestão de uma frota de táxis, utilizando algoritmos de procura em grafos para minimizar o tempo de resposta e o custo operacional. O algoritmo A* demonstrou ser o mais eficaz, encontrando a solução ótima com boa capacidade computacional. Foram ainda desenvolvidas estratégias de procura multi-destino para otimizar as procuras. A simulação permitiu a inclusão de outras variáveis, como o trânsito, o tipo de veículo, e as preferências do cliente, resultando num sistema realista, eficiente e que cumpre os objetivos propostos.